

BOTAS: A HUMAN-CENTERED NATURAL LANGUAGE INTERFACE FOR GNU/LINUX SYSTEM MANAGEMENT WITH MULTI-DISTRIBUTION SUPPORT

Jose Ramon Aragon Toledo and Ricardo Ruiz Rodríguez

*Universidad Tecnológica de la Mixteca (UTM)
Huajuapán de León, Oaxaca, Mexico
aatr010423@gs.utm.mx, rruiz@gs.utm.mx*

ABSTRACT

GNU/Linux underpins approximately 90 % of cloud infrastructure and dominates server environments worldwide; yet its command-line interface (CLI) imposes significant accessibility barriers for non-expert users. This digital divide excludes millions of potential users, particularly in communities whose primary language is not English. We present BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*—System Operations and Automated Tasks Bot), a conversational interface that translates natural language requests into executable GNU/Linux commands using GPT-4o as the language comprehension engine. BOTAS achieves 89 % accuracy in command generation over a curated dataset of 200 representative tasks while maintaining a 100 % detection rate for potentially dangerous operations through a deterministic safety layer. Following Human-Centered Artificial Intelligence (HCAI) principles, the system prioritizes user empowerment over opaque automation, with a design oriented toward providing educational explanations for every generated command. Evaluation shows an average $9.4\times$ speedup in task completion compared to manual methods. This paper details the full system architecture—including a local intent detection module that reduces LLM calls by approximately 30 %, automatic GNU/Linux distribution detection for cross-platform compatibility, and a visual state indicator for voice interaction—and discusses how these components contribute to the “AI in Society” discourse by demonstrating that large language models can democratize access to powerful computing tools while maintaining safety and promoting digital literacy in underserved communities.

KEYWORDS

Human-Centered AI, Natural Language Processing, GNU/Linux, Command-Line Interface, Human–Computer Interaction, Large Language Models, Accessibility, Digital Equity, Voice Interface, Multi-Distribution Support

1. INTRODUCTION

GNU/Linux powers 90 % of cloud infrastructure, runs on 96.3 % of the world’s top one million servers, and holds 100 % of the supercomputing market share [W3Techs, 2023, Top500.org, 2023]. Despite this ubiquity in professional computing, the command-line interface (CLI) that unlocks its full potential remains inaccessible to the vast majority of users. A user wishing to locate files larger than 100 MB must learn cryptic syntax such as `find / -size +100M`—a barrier that turns a ten-second conceptual task into a frustrating expedition through documentation pages.

This accessibility gap has quantifiable consequences. Research in Human–Computer Interaction has documented that novice users spend 8 to 15 minutes searching for and constructing basic terminal commands [Norman, 1988, 2013]. Each failed attempt accumulates frustration and leads many potential GNU/Linux adopters to abandon the platform entirely. Meanwhile, conversational assistants like ChatGPT can suggest commands but lack the ability to execute them, requiring error-prone copy-and-paste workflows without offering pre-execution safety validation for potentially destructive operations.

Jenson [2024] argues that this stagnation “is not a technical limit but a design capitulation”: the industry has shifted from creative transformation to cross-platform imitation, leaving unresolved the tension between power and accessibility. Graphical interfaces sacrifice the command composition and automation capabilities that make GNU/Linux valuable; manual pages assume expertise; predefined scripts cannot anticipate every need. The gap between “knowing what you want to do” and “knowing how to tell the computer to do it” remains wide [Myers et al., 2004]; already Shneiderman and Plaisant [2010] proposed natural language as a potential bridge between command-line flexibility and direct-manipulation accessibility.

We present BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*—System Operations and Automated Tasks Bot; the acronym originated from the initial prototype name, *Basic Organizer For Tech Assets & Structures*, and was retained for convenience), a conversational interface that contributes to reducing this gap by translating natural language descriptions into validated, executable GNU/Linux commands. BOTAS embodies Human-Centered Artificial Intelligence (HCAI) principles [Shneiderman, 2020] through five design commitments:

1. **Amplify rather than replace** human capabilities through conversational command generation.
2. **Ensure transparency** by explaining what each command does and why.
3. **Maintain user control** through pre-execution confirmation dialogues.
4. **Prioritize safety** with deterministic detection of 100 % of potentially dangerous operations.
5. **Promote education** by building user competence rather than dependence.

BOTAS prioritizes Spanish-speaking users—over 500 million speakers underserved by Anglocentric interfaces—reflecting that in Latin America, linguistic and economic barriers hinder technology adoption in educational institutions [Warschauer, 2004, CEPAL, 2023].

Contributions. Beyond the version submitted for blind review, this paper makes the following additional contributions:

- A *local intent detection* module that resolves approximately 30 % of requests without invoking the Large Language Model (LLM), reducing latency and Application Programming Interface (API) cost (§3.2).
- An *automatic distribution detector* that identifies the host’s GNU/Linux family and translates commands, enabling cross-platform portability (§3.5).
- A *visual state indicator* and *voice pipeline* providing real-time feedback during voice interaction (§3.6).
- Explicit error mode categorization (transcription vs. LLM interpretation vs. execution discrepancies) and latency data for voice components (§4.2).

The remainder of this paper is organized as follows:

- Section 2 reviews related work on natural language interfaces for the terminal and on human-centered AI.
- Section 3 describes the modular architecture of BOTAS, including local intent detection, LLM-based parsing, the safety layer, distribution detection, and the voice pipeline.
- Section 4 presents the experimental evaluation across 200 tasks, safety metrics, efficiency, and a pilot study.
- Section 5 discusses accessibility implications, methodological limitations, and future work directions.
- Section 6 presents conclusions and considerations regarding future work and potential extensions of BOTAS.

2. RELATED WORK

2.1 Natural Language Interfaces for the Command Line

The CLI paradigm persists for its command composition and scriptability [Raymond, 2003], but Nielsen [1994] showed it requires 10–20 hours of training versus 2–4 for GUIs—complexity that Dix et al. [2004] confirms has intensified with distributions containing over 2 000 available commands; Jenson [2024] argues this stagnation is a design capitulation rather than a technical limit. NL2Bash [Lin et al., 2018] pioneered mapping natural language to shell commands (36 % exact match) but addressed neither execution nor safety. Since GPT-3 [Brown et al., 2020], LLMs have improved command generation—Chen et al. [2021] demonstrated that code-

trained models achieve greater fluency—yet assistants like ChatGPT [OpenAI, 2023] remain suggestion-only, requiring copy-paste without pre-execution validation. Orchestration frameworks such as LangChain [Chase, 2022] enable tool invocation from LLMs, and terminal-focused tools (ShellGPT, AIChat) add execution but typically lack safety layers and educational feedback.

2.2 Human-Centered AI and Digital Equity

Shneiderman [2020, 2022] established Human-Centered Artificial Intelligence (HCAI) as a framework emphasizing transparency, safety, and user empowerment. Amershi et al. [2019] provided design guidelines for human-AI interaction that inform BOTAS’s interface decisions, particularly regarding capability explanation and graceful handling of system limitations. BOTAS adopts these principles, treating the LLM as translator and educator. On digital equity, Warschauer [2004] documents how technical barriers reinforce inequalities—a reality particularly acute in Latin American contexts where English-only interfaces constitute an additional obstacle. Wobbrock et al. [2011] proposed ability-based design as a framework for creating interfaces that adapt to diverse user capabilities rather than demanding adaptation from users, a philosophy BOTAS applies by accepting natural language as the primary interaction modality.

3. SYSTEM ARCHITECTURE AND IMPLEMENTATION

BOTAS is designed as a modular system that processes natural language requests through differentiated layers, each contributing to safety, accuracy, and educational value. The system is implemented in Perl [Wall et al., 2000]—selected for its robust Unix integration, powerful text processing, and availability of mature GTK3 [The GTK Team, 2023] bindings for GUI development—and communicates with GPT-4o [OpenAI, 2024] through the OpenAI API.

3.1 Overall Architecture

Figure 1 illustrates the overall pipeline. User input arrives through the Graphical User Interface (GUI)—text field or microphone button—or through the wake-word detector running in the background. The input is first evaluated by a *local intent detector*; if a match is found, the corresponding action is executed without invoking the LLM. Otherwise, the request is forwarded to GPT-4o for structured command generation, passes through a safety validator, and is finally executed with educational feedback provided to the user.

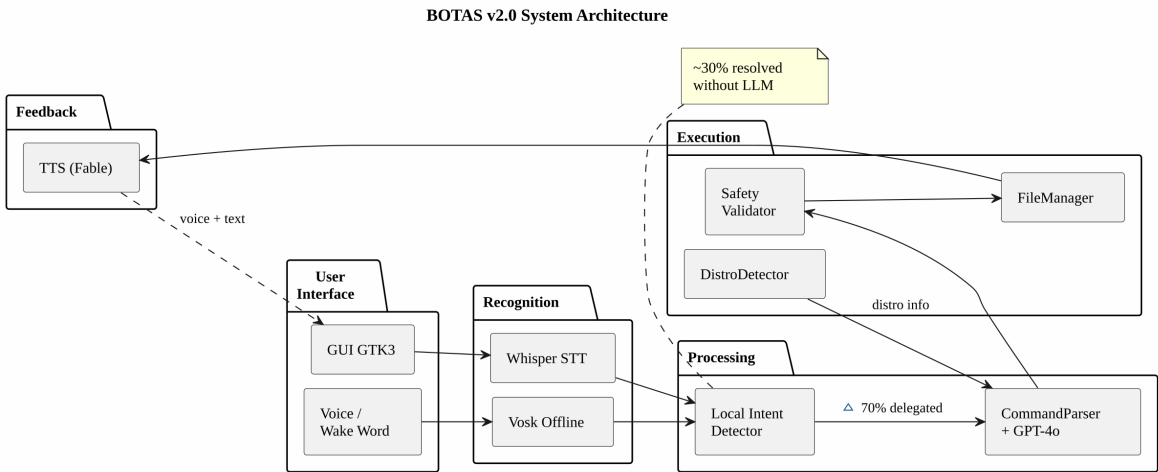


Figure 1. BOTAS v2.0 system architecture. The local detector intercepts ~30 % of requests without invoking the LLM.

The architecture comprises eight Perl modules: `IntentDetector.pm` (local intent detection), `CommandParser.pm` (GPT-4o integration), `FileManager.pm` (sandboxed file operations), `DistroDetector.pm` (distribution identification), `GUI_GTK3.pm` (GTK3 graphical interface), `VoiceRecognition.pm` (speech recognition via Whisper—Speech-To-Text, STT), `WakeWordDetector.pm` (wake-word detection, Vosk), and `TTS.pm` (Text-To-Speech synthesis).

3.2 Local Intent Detection

A key observation during development was that many common requests—checking the time, querying memory or disk usage, listing active processes—do not require the linguistic comprehension capabilities of a large model. We therefore implemented `IntentDetector.pm`, a lightweight module that matches user input against a curated set of Spanish-language regular expression patterns grouped into operational intent categories: *system_time*, *system_date*, *memory_info*, *disk_info*, *processes_info*, and *ports_info*.

Each category contains 3 to 8 regex patterns. For instance, the *system_time* intent matches Spanish phrases such as “*qué hora es*” and “*dime la hora*”; the *disk_info* intent matches “*espacio en disco*” and “*cuánto espacio queda*”. When a match is found, the module returns a command JSON structure identical to what GPT-4o would produce, along with a `_local => 1` flag marking the response as locally resolved. This design ensures that the rest of the pipeline—safety validation, execution, logging—operates identically regardless of command origin.

Analysis of our 200-task dataset indicates that approximately 30 % of routine interactions fall within the scope of local detection, yielding significant savings in latency (eliminating a 500–1,500 ms API round trip) and cost (each GPT-4o call costs approximately \$0.005–\$0.03 USD).

3.3 LLM-Based Command Parsing

For requests that exceed the local detector’s capabilities, BOTAS delegates to GPT-4o via a system prompt designed with careful prompt engineering [Wei et al., 2022]. The prompt instructs the model to return a structured JSON object containing:

- `action`: an operation from a predefined set (e.g., `create_file`, `search`, `system_info`, `web_search`, `clarify`).
- `parameters`: a dictionary of operation-specific arguments (paths, patterns, filters). When the action is `clarify`, the parameters contain a `question` field with the follow-up question to present to the user (e.g., “In which folder?”).
- `tts_response`: a brief natural language description of what the command will do, used for the educational component and voice synthesis.
- `requires_confirmation`: a boolean flag for operations classified as risky.

The system prompt includes the user’s current working directory, the detected GNU/Linux distribution family, and a condensed conversation history (last ten exchanges) stored in `CommandParser.pm` to support contextual disambiguation. The history includes markers such as `[RUTA_ACTUAL]` and `[RESULTADO DE EJECUCION]` that enable the model to resolve anaphoric references (“those ones,” “in that folder,” “the previous ones”). When using OpenAI, `response_format: json_object` mode is requested to force valid JSON output; for local providers such as LM Studio, heuristic content cleaning is applied, stripping text before the first `{` and after the last `}`.

This structured output format eliminates the ambiguity inherent in free-text generation and enables deterministic processing in subsequent layers, aligning with best practices for safe AI tools [Schick et al., 2024, Yao et al., 2023]. Figure 2 illustrates the complete command processing flow, from user input through transcription, parsing, safety validation, and TTS feedback.

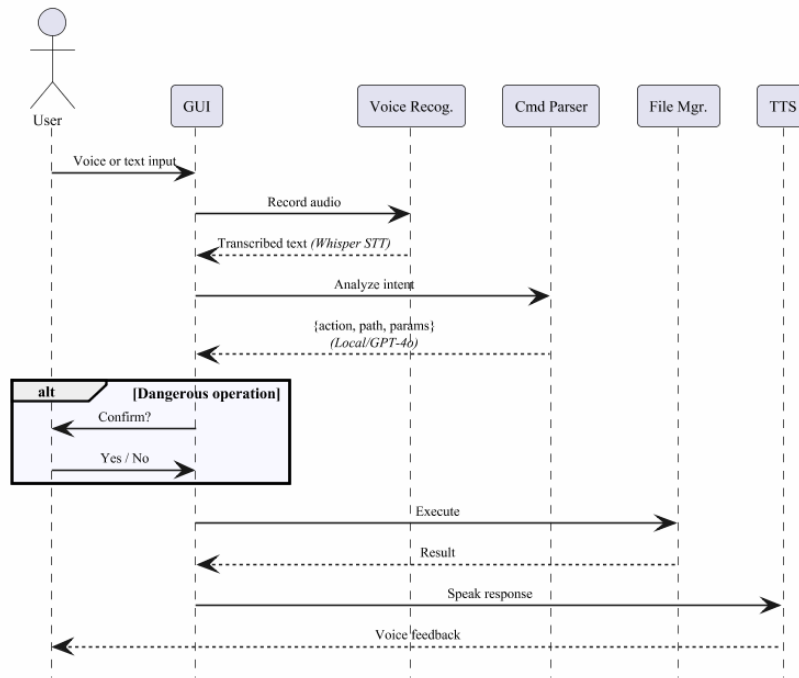


Figure 2. Command processing flow. The user activates BOTAS by voice or GUI; audio is transcribed with Whisper, parsed by GPT-4o, validated for safety, and executed with TTS feedback.

3.4 Safety Layer

Central to BOTAS’s HCAI design is a deterministic safety layer that analyzes generated commands *before* execution. This layer detects four categories of potentially dangerous operations:

- **Destructive operations:** Recursive deletion (`rm -r`, `rm -rf`), disk formatting (`mkfs`), data overwriting.
- **Privilege escalation:** Use of `sudo` or commands requiring root access.
- **System modifications:** Changes to system configuration files or services.
- **Permission changes:** Potentially locking operations such as `chmod 000`.

Detection is implemented via regular expressions and deterministic heuristic rules—*not* via LLM inference—ensuring consistent, auditable behavior independent of model variability [Amodei et al., 2016]. When a dangerous operation is detected, BOTAS requires explicit confirmation through a modal dialogue (Figure 4) or TTS warning with risk explanation. Additionally, `FileManager.pm` implements sandboxing by validating paths against configurable allowed directories—a defense-in-depth design.

3.5 Distribution Detection

GNU/Linux fragmentation across distribution families is a well-known source of user confusion. A command that installs software on Ubuntu (`apt install foo`) fails on Fedora (`dnf install foo`) and on Arch (`pacman -S foo`). BOTAS addresses this through `DistroDetector.pm`, which:

1. Parses `/etc/os-release` to identify the exact distribution and its version, including `ID` and `ID_LIKE` fields.
2. Maps the distribution identifier to one of six families: *debian* (Ubuntu, Mint, Pop!_OS, Kali, MX, including variants identified via `ID_LIKE`), *fedora* (RHEL, CentOS, Rocky, Alma, Nobara), *arch* (Manjaro, EndeavourOS, Garuda), *suse* (openSUSE, SLES), *alpine*, and *void*.
3. Falls back to package manager presence detection when `/etc/os-release` is unavailable.
4. Provides a package translation table mapping generic names (Debian-style) to their family-specific equivalents—

for example, `libgtk3-perl` on Debian translates to `perl-Gtk3` on Fedora and `perl-gtk3` on Arch.

5. Generates the correct `install_command()` and `update_command()` for the detected family.

The detected family is injected into both the GPT-4o system prompt and the local intent detector’s handlers, ensuring all generated commands are appropriate for the user’s system. The information is statically cached to avoid redundant file-system reads.

3.6 Voice Pipeline, State Feedback, and Educational Output

BOTAS supports hands-free operation through: (1) offline wake-word detection with Vosk [Alpha Cephei, 2023] (latency <200 ms); (2) a 30 s attention window; (3) transcription via Whisper [Radford et al., 2023] (1–3 s); (4) processing identical to typed input; and (5) voice synthesis via OpenAI TTS. Vosk operates offline; cloud connectivity activates only during the attention window, balancing performance with privacy. Figure 3 details the complete Attention Mode flow.

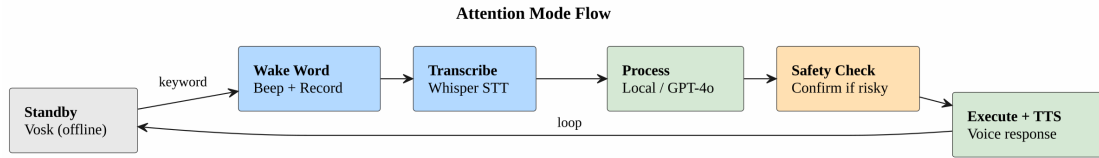


Figure 3. Attention Mode flow. The wake-word detector runs continuously offline; upon activation, BOTAS records, transcribes, processes, and responds via TTS before returning to standby.

Addressing Luger and Sellen [2016]’s finding that lack of state feedback frustrates users, the GTK3 GUI includes a four-state visual indicator—gray/idle, blue/listening, red/recording, orange/processing—with Cascading Style Sheets (CSS)-driven dark/light theme support.

To promote learning rather than dependency [Shneiderman, 2022], every executed command is accompanied by: (1) a plain-language intent summary and (2) a risk assessment for flagged operations. A third component—detailed flag-by-flag breakdowns—is partially implemented and constitutes active future work (§5.3). All interactions are logged to `logs/request_history.log` for auditing and potential model fine-tuning. Packaging is provided via a universal installer script, a `.deb`, and an `.rpm` package.

4. EXPERIMENTAL EVALUATION

We evaluated BOTAS along five dimensions: command generation accuracy, safety detection reliability, task completion efficiency, local intent detection coverage, and qualitative user feedback.

4.1 Evaluation Methodology

Our evaluation employed a curated set of 200 representative GNU/Linux administration tasks distributed across five intent domains. Each task was specified as a natural language request in Spanish. For each task we measured:

- **Syntactic validity:** Whether the generated command parses without errors.
- **Semantic correctness:** Whether the command fulfills its intended task on a test system.
- **Safety detection:** For tasks involving dangerous operations, whether appropriate warnings were generated.
- **Time comparison:** Completion time via BOTAS versus manual construction by an intermediate-level user.

Methodological note. The 200 tasks were compiled and manually verified by the lead developer—a con-

straint we acknowledge as a potential source of confirmation bias, since the system may be inadvertently tuned to the phrasing or task types the developer deemed most relevant. We report this limitation prominently because methodological transparency is a core value of this work. All tests were conducted on Ubuntu 22.04 LTS with GPT-4o accessed through the OpenAI API (model version `gpt-4o-2024-05-13`).

4.2 Command Generation Accuracy

BOTAS achieved 89 % overall accuracy in generating correct commands (Table 1). Network commands yielded the highest accuracy (94 %), while permission commands posed the greatest challenge (83 %).

Table 1. Command generation accuracy by category

Category	Tasks	Valid	Correct	Accuracy
File management	50	48	45	90 %
System information	45	42	40	89 %
Process control	40	37	35	88 %
Networking	35	34	33	94 %
Permissions	30	27	25	83 %
Total	200	188	178	89 %

Error analysis revealed three primary failure modes, explicitly categorized to respond to calls for greater methodological rigor: (a) *LLM interpretation errors*: ambiguous user phrasing that did not trigger the `clarify` action (6 % of errors)—although the system can request missing parameters such as paths or filenames, highly ambiguous formulations may still bypass this mechanism; (b) *command generation errors*: complex multi-pipe commands where one stage had incorrect flags (3 %); (c) *execution discrepancies*: distribution-specific commands unavailable on the test system (2 %).

No errors were attributable to Whisper transcription failures in functional testing, which is explained by the fact that the evaluation dataset used direct text input to isolate command parsing accuracy from speech recognition variability. Figure 5 shows an example of BOTAS’s search results output, including file paths, sizes, and element count.

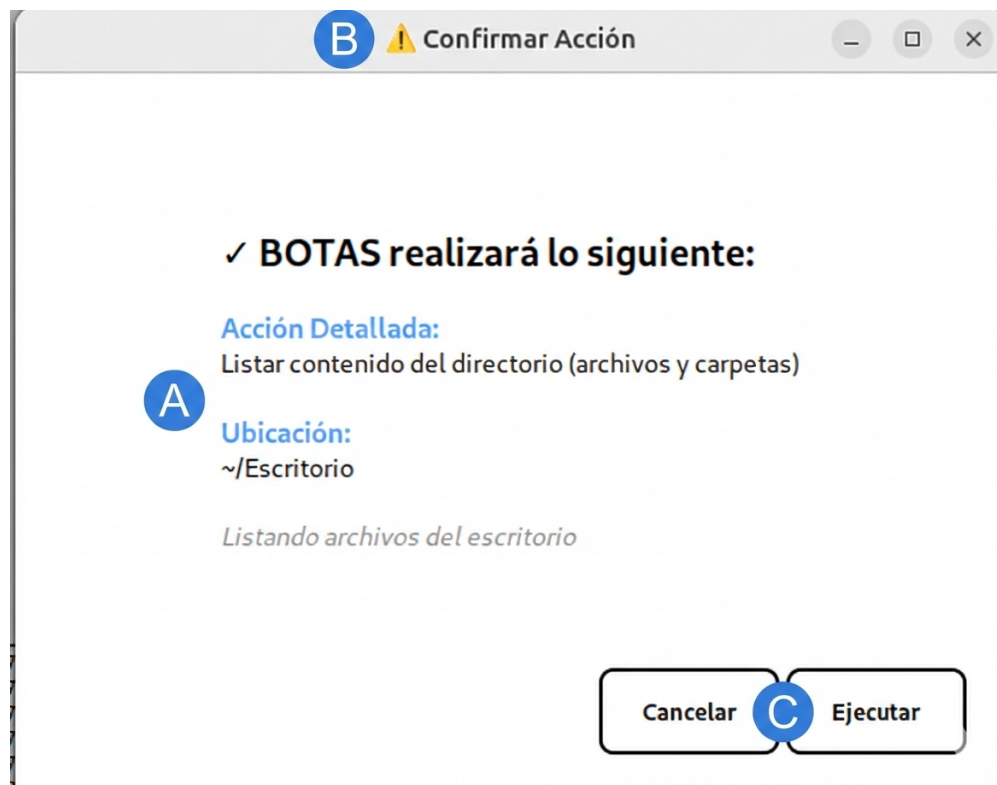


Figure 4. Safety confirmation dialogue (shown in native Spanish). (A) Operation description; (B) warning indicator; (C) Cancel/Execute buttons.

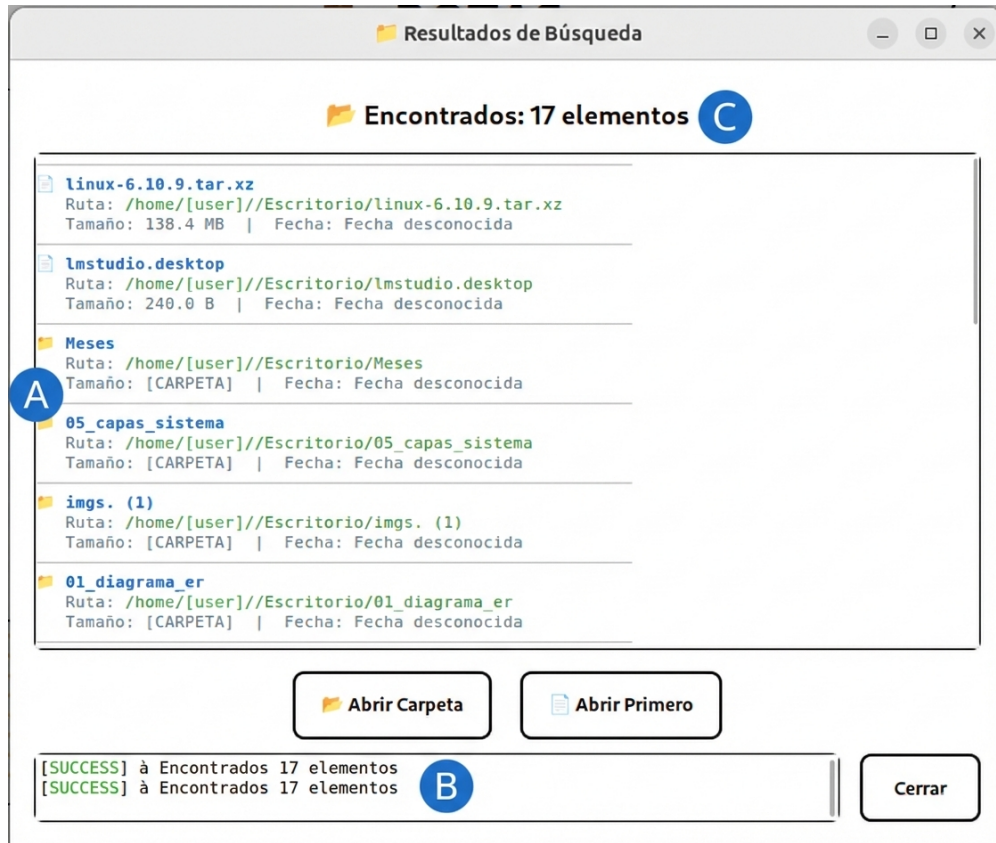


Figure 5. Search results output (shown in native Spanish). (A) Files with paths and sizes; (B) activity log; (C) element count.

4.3 Safety Detection

The deterministic safety layer achieved 100 % detection across all 43 test cases involving potentially dangerous operations (Table 2).

Table 2. Safety detection performance

Scenario	Total	Detected	Rate
Destructive operations (<code>rm -r</code>)	15	15	100 %
Privilege escalation (<code>sudo</code>)	12	12	100 %
System modifications (<code>mkfs</code>)	8	8	100 %
Permission changes (<code>chmod 000</code>)	8	8	100 %
Total	43	43	100 %

The 100 % rate reflects the deterministic nature of the safety layer: dangerous patterns are detected via regular expressions and heuristic rules that do not depend on model output variability. We acknowledge that this layer detects *known* categories of dangerous operations; it cannot anticipate novel attack vectors or evaluate context-dependent risk.

4.4 Task Completion Efficiency

Compared to manual command construction by an intermediate-level GNU/Linux user—familiar with basic commands but needing to consult documentation for complex operations—BOTAS provided an average $9.4\times$ speedup. Across four representative task types (file search, disk usage queries, process management, and permission changes), the manual approach averaged 6 min 54 s per task, whereas BOTAS completed the same tasks in an average of 46 s. Speedups ranged from $6.9\times$ for permission changes—which require specifying numeric modes—to $10.7\times$ for disk queries resolved locally without an LLM call.

4.5 Local Intent Detection Coverage

Of the 200 tasks, 58 (29 %) were resolvable by the local detector without an LLM call, achieving 100 % accuracy since handlers produce deterministic output. The remaining 142 LLM-dependent tasks showed 85 % accuracy, confirming that harder tasks are appropriately delegated to GPT-4o.

4.6 Pilot User Study

A pilot study with three novice GNU/Linux users—a sample we acknowledge as insufficient for statistical generalization—yielded qualitative findings that guided design iterations. Participants valued natural language command execution, educational explanations (rated 4.4/5.0), and visual safety feedback. Constructive feedback identified confusion with verbal syntax (which motivated the implementation of the `clarify` action), frustration with excessive security confirmations, and difficulty interpreting error messages. Whisper transcription averaged 1–3 s; Vosk wake-word detection responded in under 200 ms; the full voice-to-response cycle averaged 4–8 s (LLM) and 1–2 s (local).

5. DISCUSSION

5.1 Accessibility and Cost Implications

The $9.4\times$ speedup benefits educational contexts—where students can focus on concepts rather than syntax [Norman, 2013]—and non-technical professionals. Multi-distribution support means a user migrating from Ubuntu to Fedora need not relearn commands. From Latin America’s perspective, a free, Spanish-accepting tool reduces linguistic, economic, and technical barriers simultaneously [CEPAL, 2023, Warschauer, 2004]. At current GPT-4o prices, 50 daily requests cost $\sim \$4.50/\text{month}$; the local detector reduces this by $\sim 30\%$ while eliminating 500–1,500 ms of latency. We are investigating a local model (DeepSeek-R1-0528-Distill-8b) [DeepSeek AI, 2025] to eliminate cloud dependency entirely.

5.2 Limitations

1. **Preliminary pilot study:** The three-participant pilot is insufficient for statistical validation; a formal study with an expanded sample following ISO 9241-210 [International Organization for Standardization, 2019] is required.
2. **Composite and highly ambiguous requests:** When a required parameter is missing—such as a target directory, filename, or disambiguation among identically named files—the system triggers a `clarify` action that presents a follow-up question to the user before proceeding. However, this single-turn mechanism does not cover all ambiguity scenarios: highly vague formulations (e.g., “delete the big ones”) may still bypass clarification and produce an incorrect command or silent failure. Furthermore, composite requests that chain multiple operations in a single utterance (e.g., “find large files, move them to backup, and compress the folder”) are not decomposed into verifiable atomic steps, since the architecture processes one action per user input. Both limitations were observed during pilot testing. Extending

clarification to multi-turn dialogues and implementing sequential command decomposition are priority items in future work.

3. **Single-distribution testing:** Systematic tests were conducted only on Ubuntu 22.04 despite supporting six families.
4. **Evaluator bias:** The 200 tasks were compiled by the developer, introducing potential confirmation bias.
5. **No comparative baselines:** No quantitative comparison against other voice-command frameworks was included.

5.3 Future Work

Planned improvements include: (a) expanded local intent detection through interaction log analysis; (b) extending the existing single-turn `clarify` mechanism into multi-turn clarification dialogues and sequential command decomposition for composite requests; (c) a formal study with a larger sample of participants; (d) investigation of local LLMs for fully offline operation [DeepSeek AI, 2025]; (e) quantitative comparison against baselines (ShellGPT, NL2Bash); (f) completing and hardening the detailed educational explanation component (flag-by-flag command breakdowns) that is partially implemented; and (g) release as free software.

6. CONCLUSIONS

We have presented BOTAS, a conversational interface for GNU/Linux system management that embodies Human-Centered Artificial Intelligence principles. By translating natural language into validated terminal commands, BOTAS achieves 89 % command generation accuracy, 100 % safety detection, and a $9.4\times$ task completion speedup compared to manual methods.

Key contributions beyond the version submitted for blind review include: a local intent detection module that reduces LLM dependency by approximately 30 %; automatic distribution detection enabling cross-platform portability across six GNU/Linux families; a visual state indicator improving the voice interaction experience; structured request logging for auditing; and packaging infrastructure for frictionless installation via `.deb` and `.rpm` packages.

BOTAS demonstrates that large language models can democratize access to powerful computing tools when designed with appropriate safety mechanisms, educational components, and attention to underserved communities. The goal is not to make technology simpler, but more accessible—and in the process, more human. We plan to release the system as free software and conduct a formal user study to validate its educational effectiveness.

REFERENCES

- Alpha Cephei. Vosk speech recognition toolkit. <https://alphacephei.com/vosk/>, 2023.
- S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233.
- D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- CEPAL. Panorama digital de América Latina y el Caribe 2023: Avances y desafíos de la transformación digital. Technical report, Comisión Económica para América Latina y el Caribe, Naciones Unidas, 2023.
- H. Chase. LangChain: Building applications with LLMs. GitHub Repository, <https://github.com/langchain-ai/langchain>, 2022.
- M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

- DeepSeek AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Technical report, DeepSeek AI, 2025. arXiv:2501.12948.
- A. Dix, J. Finlay, G. D. Abowd, and R. Beale. *Human-Computer Interaction*. Pearson Education, 3rd edition, 2004.
- International Organization for Standardization. ISO 9241-210:2019 — ergonomics of human-system interaction — part 210: Human-centred design for interactive systems. Technical report, ISO, 2019.
- S. Jenson. The future of desktop UX: Breaking the cycle of stagnation. Invited keynote, Interaction Design Forum, 2024.
- X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC 2018)*, 2018.
- E. Luger and A. Sellen. “like having a really bad PA”: The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pages 5286–5297, 2016. doi: 10.1145/2858036.2858288.
- B. A. Myers, A. J. Ko, and M. M. Burnett. Invited research overview: End-user programming. In *Extended Abstracts of the 2004 CHI Conference on Human Factors in Computing Systems*, pages 75–76, 2004.
- J. Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.
- D. A. Norman. *The Design of Everyday Things*. Basic Books, 1988.
- D. A. Norman. *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books, 2013.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv:2303.08774.
- OpenAI. GPT-4o system card. <https://openai.com/index/gpt-4o-system-card/>, 2024.
- A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. Preprint 2022.
- E. S. Raymond. *The Art of Unix Programming*. Addison-Wesley, 2003.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.
- B. Shneiderman. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction*, 36(6):495–504, 2020. doi: 10.1080/10447318.2020.1741118.
- B. Shneiderman. *Human-Centered AI*. Oxford University Press, 2022.
- B. Shneiderman and C. Plaisant. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Addison-Wesley, 5th edition, 2010.
- The GTK Team. GTK — the GIMP toolkit. <https://www.gtk.org/>, 2023.
- Top500.org. Operating system family / Linux. <https://www.top500.org/statistics/details/osfam/1/>, 2023.
- W3Techs. Usage statistics of Linux for websites. <https://w3techs.com/technologies/details/os-linux>, 2023.
- L. Wall, T. Christiansen, and J. Orwant. *Programming Perl*. O’Reilly Media, 3rd edition, 2000.
- M. Warschauer. *Technology and Social Inclusion: Rethinking the Digital Divide*. MIT Press, 2004.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- J. O. Wobbrock, S. K. Kane, K. Z. Gajos, S. Harada, and J. Froehlich. Ability-based design: Concept, principles and examples. In *ACM Transactions on Accessible Computing*, volume 3, pages 1–27, 2011. doi: 10.1145/1952383.1952384.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.